

Introduction to Regression

Part B: Error Evaluation and Model Comparison



Daniel More
Sao Paulo

Pavlos Protopapas, Kevin Rader, Chris Gumb

Lecture Outline

Part A: Statistical Modeling

k-Nearest Neighbors (kNN)

Part B: Error Evaluation and Model Comparison

How do we evaluate our model?

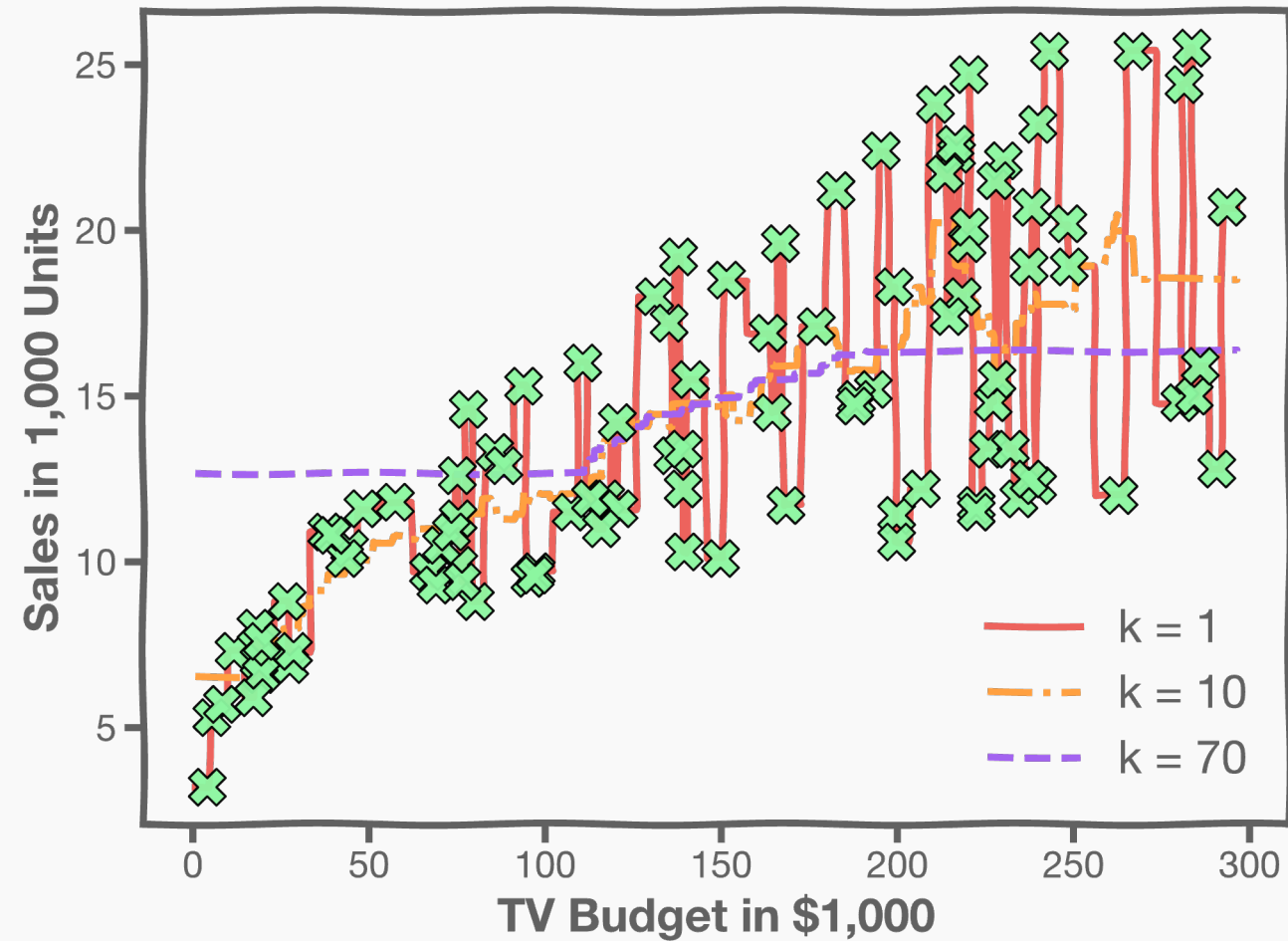
How do we choose from two different models?



<https://xkcd.com/833/>

k-Nearest Neighbors - kNN

We have tested various models using different k-values on the data.



Choices for model



Which model do you think is the best?

Options:

A. $k = 1$

B. $k = 10$

C. $k = 70$

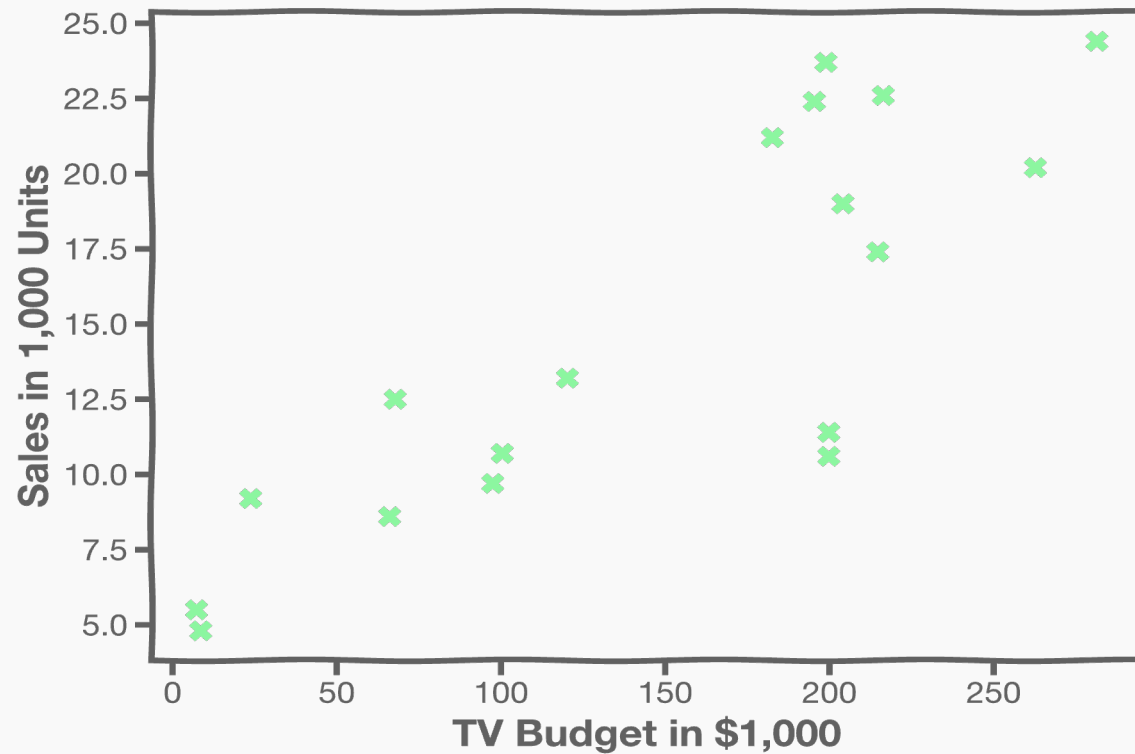
D. $k = 15$

E. 🙋

Error Evaluation

Error Evaluation

We need to **define** what we mean by **best**. To do so, we start with our data.



Error Evaluation

We first **withhold** a portion of the data from the model; this process is called **train-validation-test** split.

Train Set

The data that we use to **train** our model to **estimate**, $\hat{y}$. x



Validation Set

The data that we use to **evaluate** our model's performance **when choosing different models**.



Test Set

The data that we use **at the very end** to **evaluate** model's performance on **unseen data**.



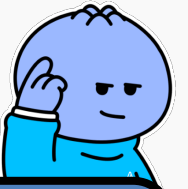
Error Evaluation

The data is randomly shuffled.

We first **withhold** a portion of the data from the model; this process is called **train-validation-test** split.

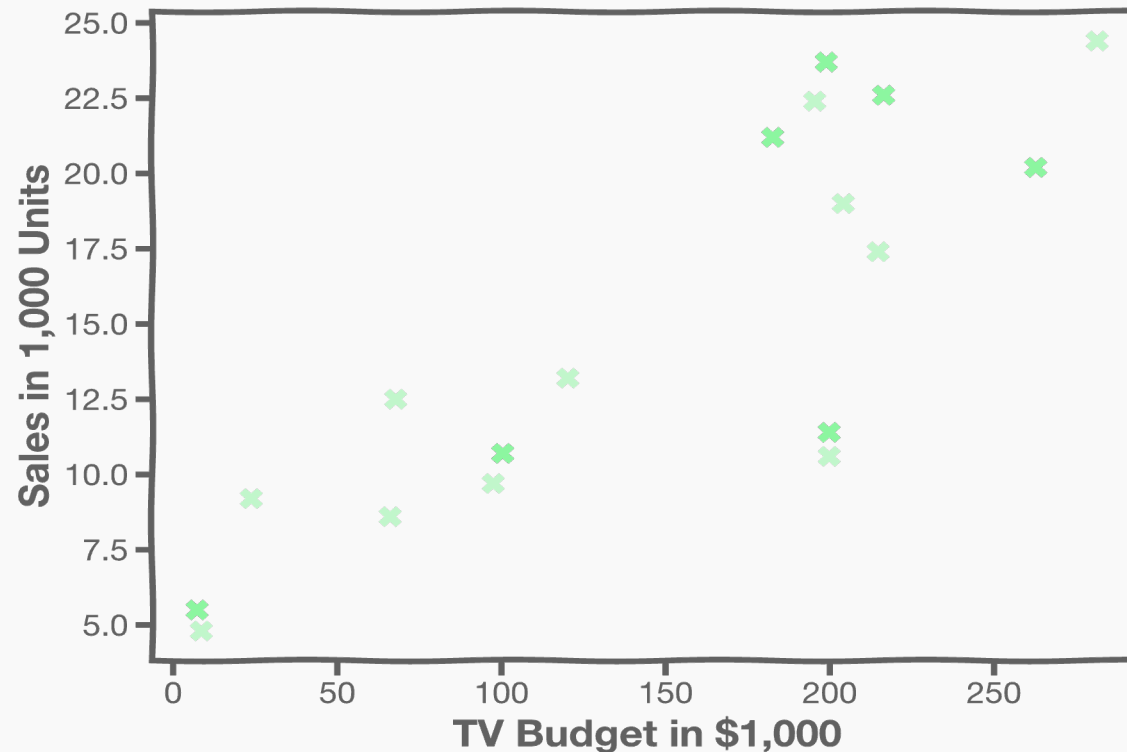
In scikit-learn, we only get a `train_test_split()` function

So, to get a validation set, we split once for train/test, then split the training data again to carve out validation.



Error Evaluation

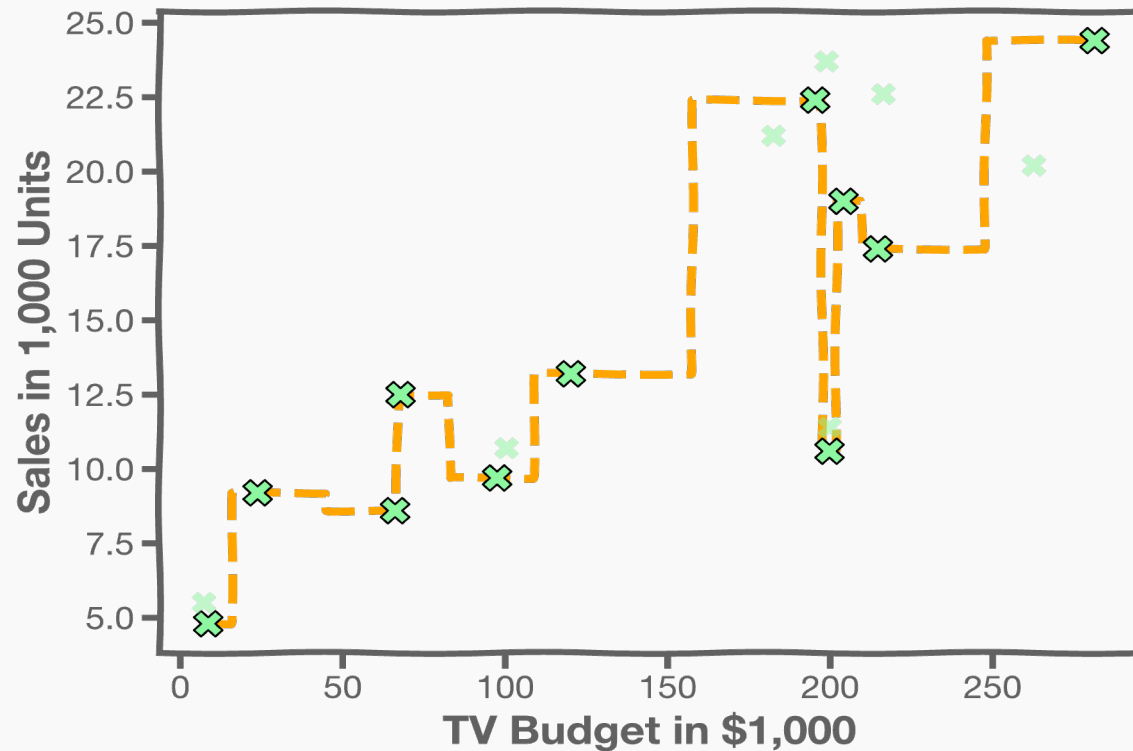
We first **withhold** a portion of the data from the model; this process is called **train-validation-test** split.



We use the **training** set to **estimate** $\hat{y}$.

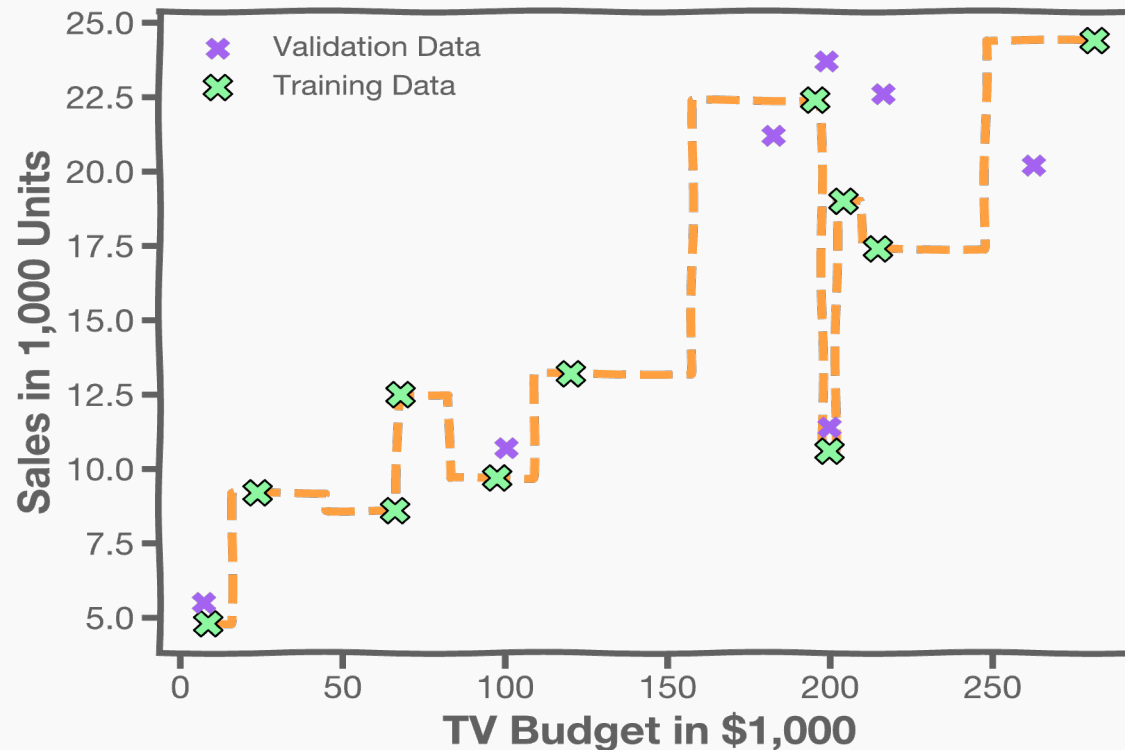
Error Evaluation

Estimate $\hat{y}'$'s values for all the data points in the training set when $k=1$.



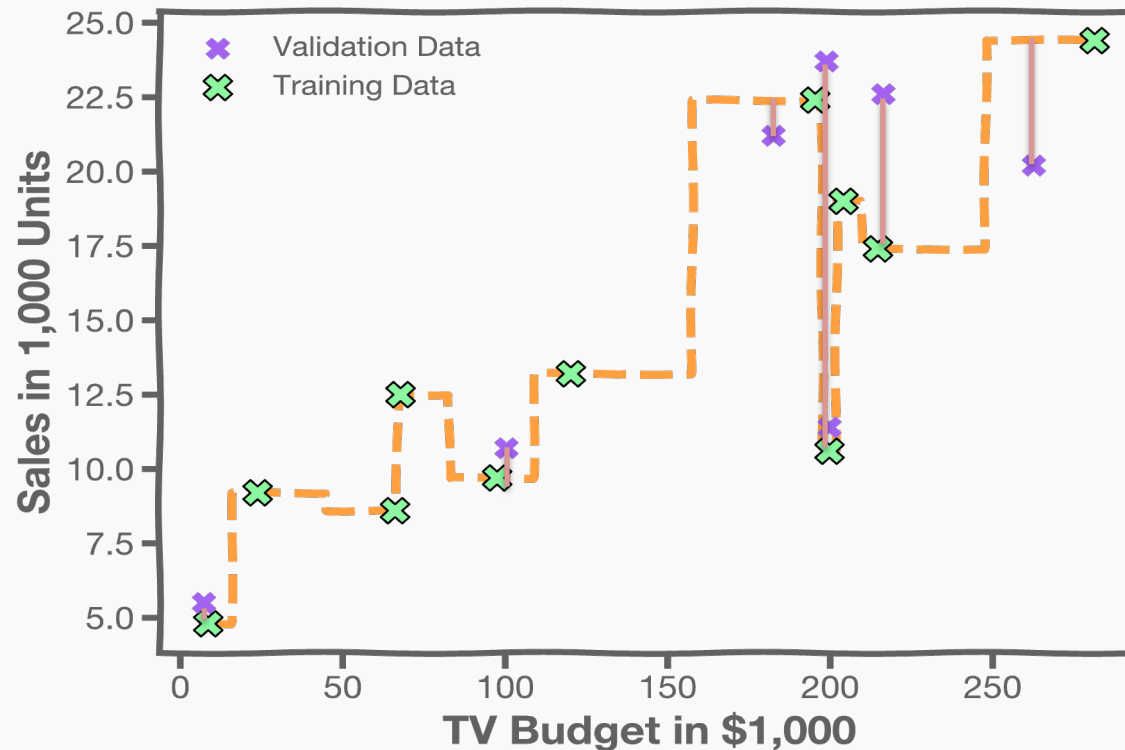
Error Evaluation

Now, we examine the data that was not used for estimating $\hat{y}$, the **validation data** represented by purple crosses.



Error Evaluation

And we calculate the **residuals** $(y_i - \hat{y}_i)$.



For each observation (x_n, y_n) , the **absolute residuals**, $r_i = |y_i - \hat{y}_i|$ quantify the error at each observation point.

Error Evaluation

To quantify the performance of a model, we **aggregate** the errors. This aggregated value is commonly referred to as the **loss**, **error**, or **cost function**.

A widely used **loss function** for quantitative outcomes is the **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Note: Loss and cost function refer to the same thing. Cost usually refers to the total loss where loss refers to a single training point.

Error Evaluation

Caution: MSE is not the only valid, or necessarily the best, loss function for all scenarios.

Other choices for loss function:

1. Max Absolute Error
2. Mean Absolute Error
3. Huber Loss

We will motivate MSE when we introduce probabilistic modeling.

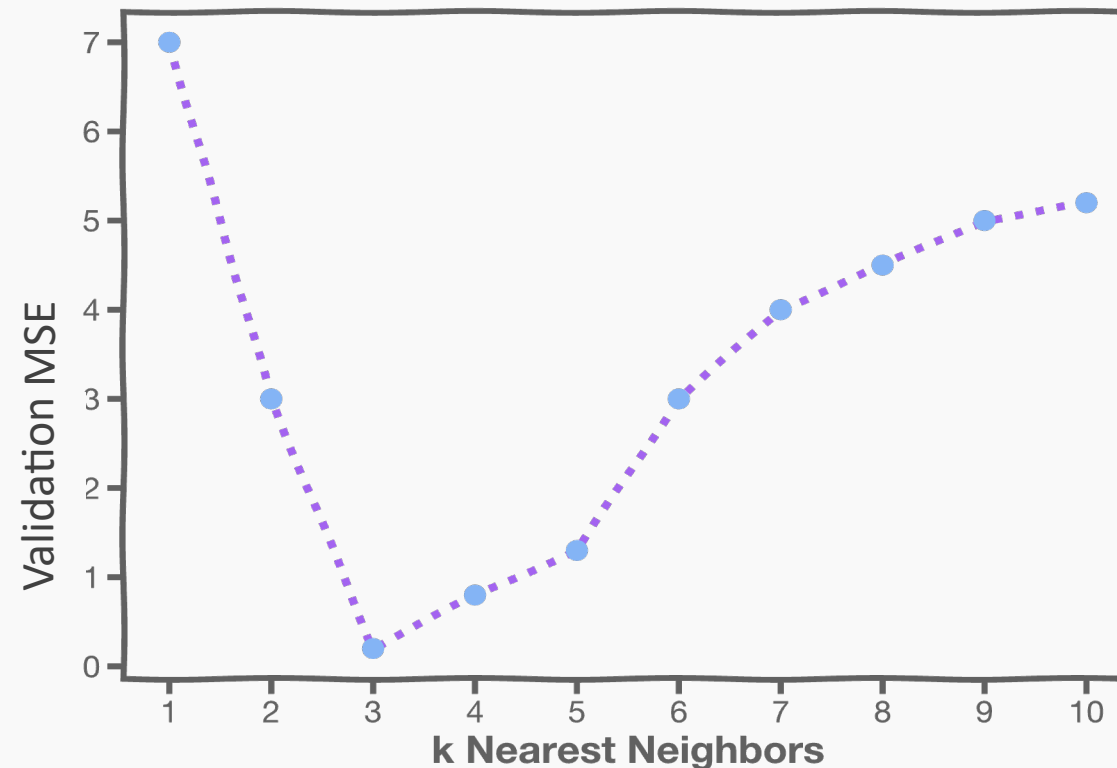
Note: The square **R**oot of the **M**ean of the **S**quared **E**rrors (RMSE) is also commonly used.

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Model Comparison

Model Comparison

We repeat this process for all values of k and compare the MSEs on the test set.



Which model is the best?

Question



Which model do you think is the best now

Options:

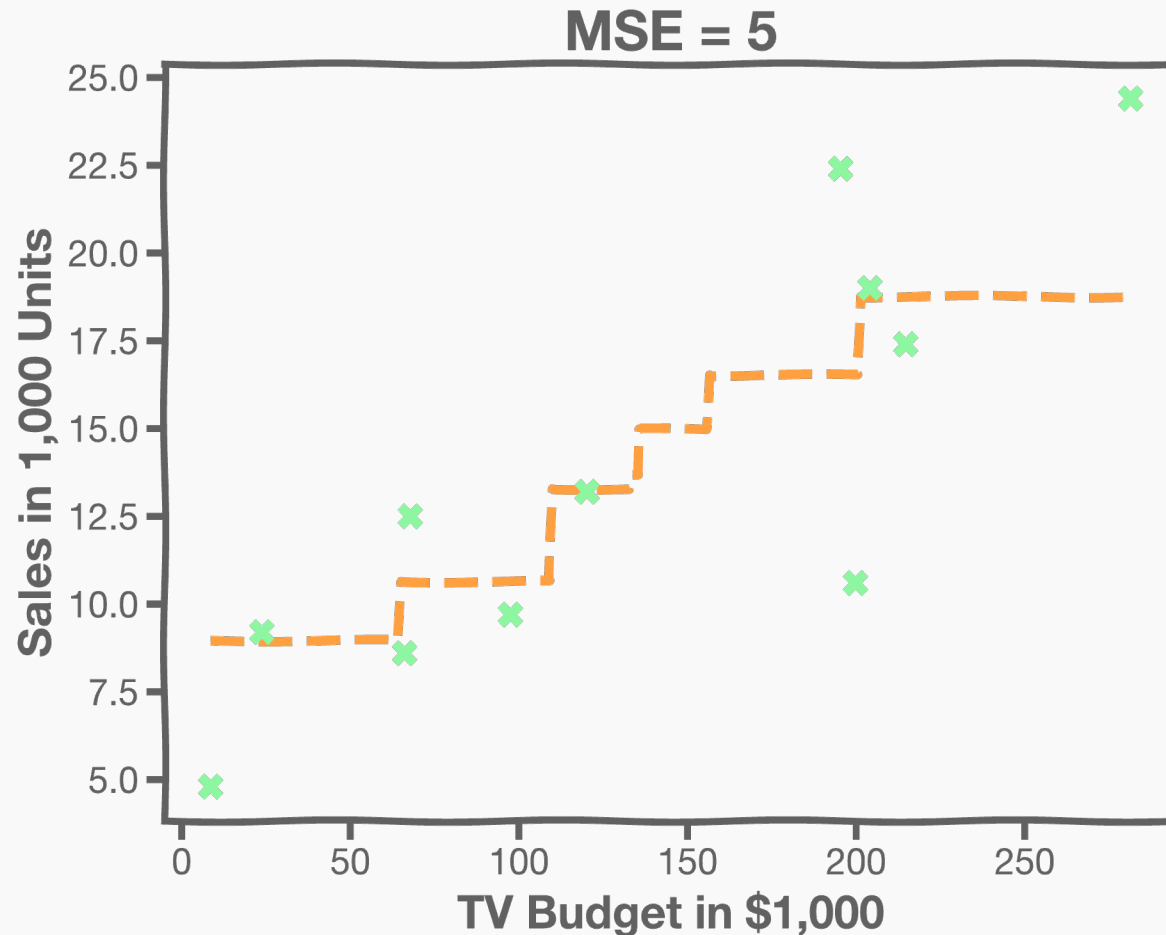
A. $k = 3$

B. $k = 3$ but why don't we experiment with different train/test splits?

Model Fitness

Model fitness

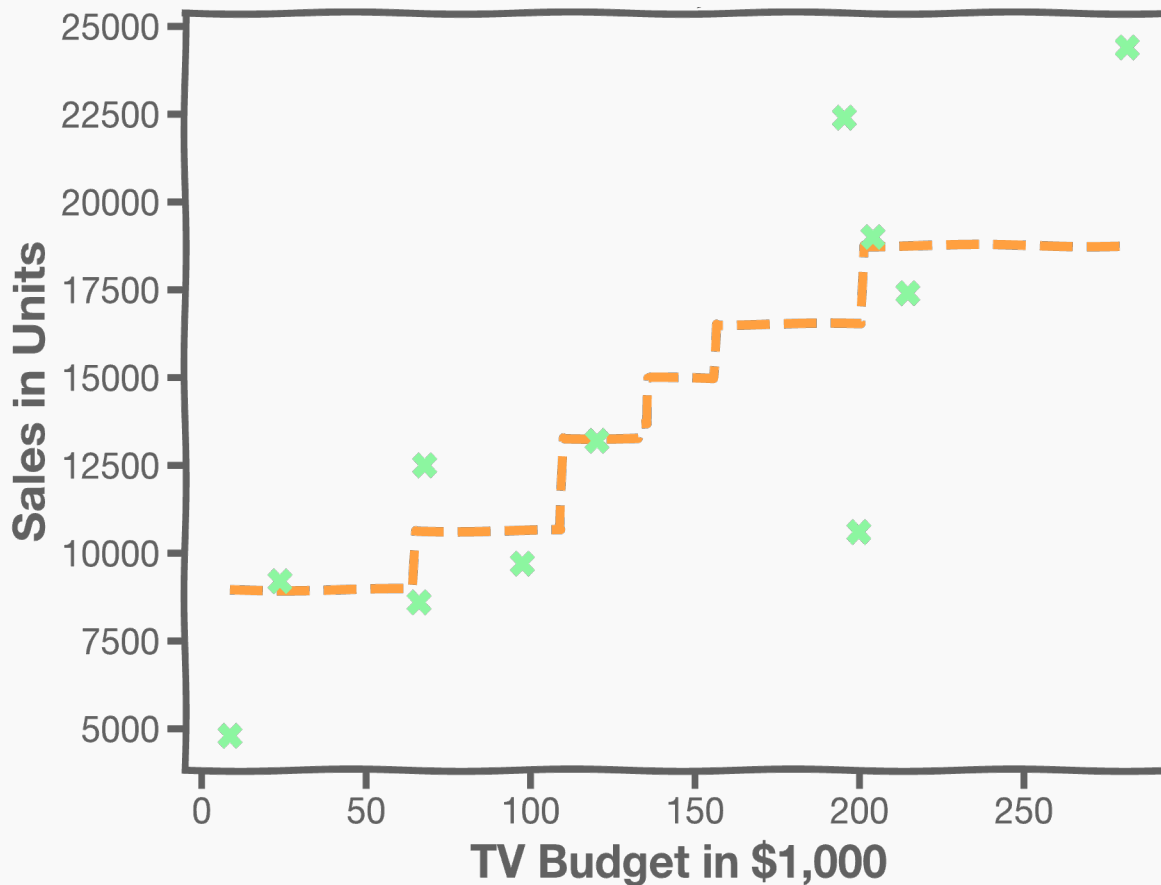
Calculate the MSE for $k = 3$ using a subset of the data.



Is MSE=5.0 good enough?

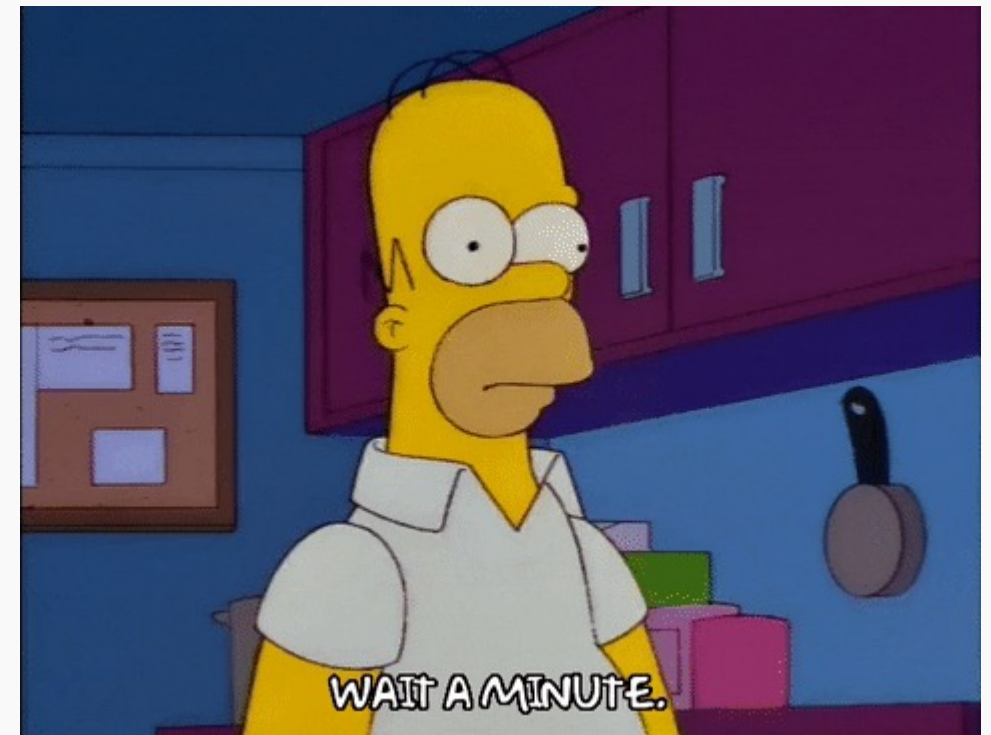
Model fitness

What would happen if we measure the ***Sales*** in single units instead of 1000 units?



MSE is now 5,004,930.

Is that good?



Model fitness

It would be more **meaningful** to compare it to a benchmark or a known value.

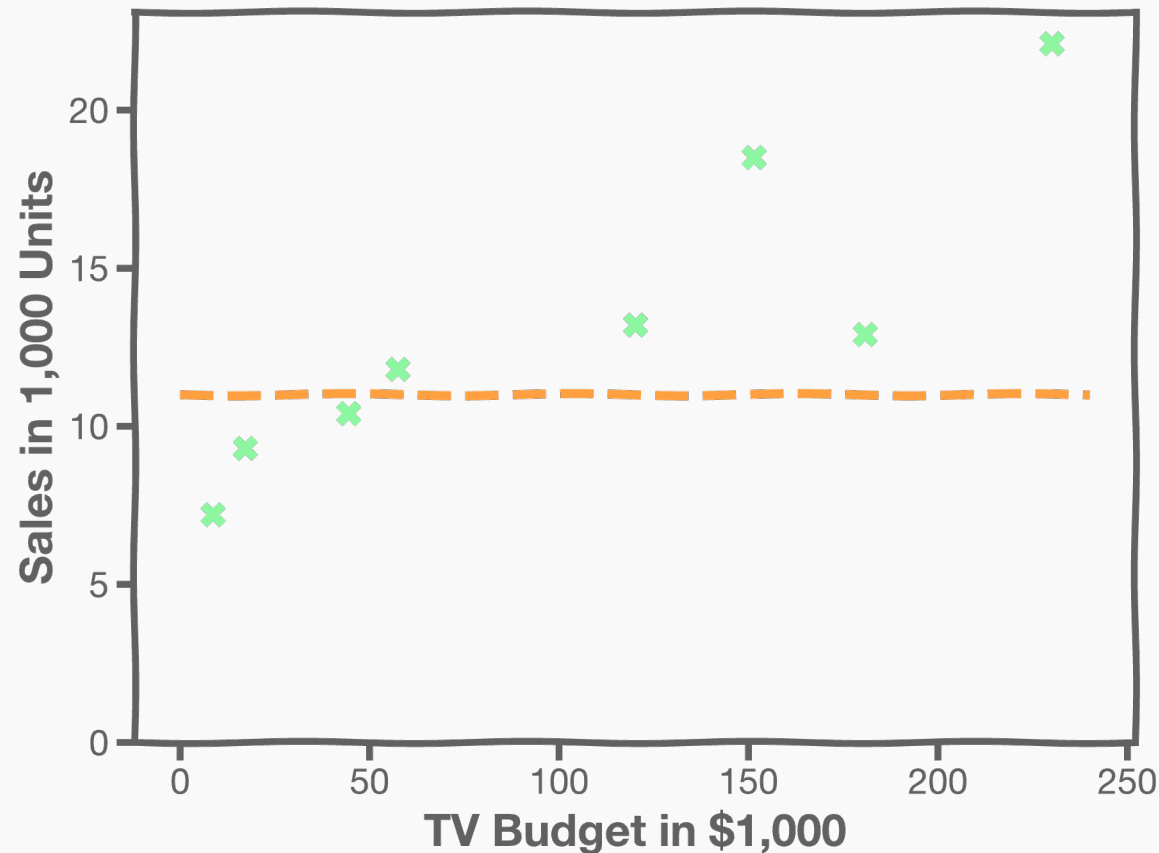
A benchmark that isn't affected by the **scale of the data**.



Model fitness

It would be more **meaningful** to compare it to a benchmark or a known value.

MSE=5.4



We will use the simplest model:

$$\hat{y} = \bar{y} = \frac{1}{n} \sum_i y_i$$

as the **worst** possible model and

$\hat{y}_i = y_i$ as the **best** possible model.

R-squared

Though is called R-squared, it is not the square of a quantity R as given in this formula.

We will use two reference models for comparison:

1. The **simplest** model, often considered the **worst** possible, where the predicted value $\hat{y}$ is the **mean** of all observations:

$$\hat{y} = \bar{y} = \frac{1}{n} \sum_i y_i$$

2. The **ideal** or best possible model, where the predicted value $\hat{y}$ is identical to the actual value y .

Using these two reference models, we define the R^2 (R-squared) value as:

$$R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (\bar{y} - y_i)^2}$$

R-squared

$$R^2 = 1 - \frac{\sum_i (\hat{y}_i - y_i)^2}{\sum_i (\bar{y} - y_i)^2}$$

- If our model is as good as the **mean value**, $\bar{y}$, then $R^2 = 0$
- If our model is **perfect**, then $R^2 = 1$
- R^2 can be **negative** if the model is worse than the average. This can happen when we evaluate the model in the **test** set.

kNN in Higher Dimensions

- So far, we've looked at kNN in 1D (a single predictor).
- In practice, kNN works with **multiple predictors (features)**.
- Distance is computed in the multi-dimensional space:

$$d \left((x_1, x_2, \dots, x_p), (x'_1, x'_2, \dots, x'_p) \right)$$

Most often: **Euclidean distance**

$$d = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}$$

The Curse of Dimensionality

- As dimensions increase:
 - Data becomes sparse.
 - “Neighbors” may be far away in high-dimensional space.
- Distances tend to even out → harder to distinguish close vs. far.
- kNN can struggle unless:
 - You have a lot of data.
 - You use **feature scaling / selection**.

Takeaway: kNN is intuitive, but high-dimensional spaces make it less effective without care.

Learning Objectives

By the end of this lecture, you should be able to:

- **Define** response and predictor variables, and represent them using the design matrix X and response vector y .
- **Explain** the difference between inference and prediction.
- **Describe** the k-Nearest Neighbors (kNN) algorithm as a **non-parametric method** that relies on data rather than fixed parameters .
- **Implement** kNN in 1D:
 - Find nearest neighbors using distance.
 - Predict outputs by averaging their values.

Learning Objectives

Evaluate kNN model performance using validation/test sets and error metrics.

Extend kNN to multi-dimensional data:

- Compute distances in p -dimensional space (Euclidean distance).
- Recognize scaling and feature importance issues.

Understand the curse of dimensionality and why kNN becomes less effective in high-dimensional spaces.

